Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Sınav sorularının çözümü: ana teoremi tersten uygulama (ekrandan, 00:06:40)
  - İstenen: çalışma zamanı $n^{\log_b a}$ olan bir yineleme bağıntısı kurmak, ağacını çizmek, her seviyedeki ve toplam işlem sayısını bulmak.
  - $T(n) = aT(n/b) + O(n^d)$ biçiminde $n^{\log_b a}$ sonucu $a > b^d$ durumunda çıkar. Bu eşitsizliği sağlayan somut a, b, d seçilmelidir; harflerle çizilen ağaç kabul edilmedi.
  - Hocanın örneği: $T(n) = 3T(n/3)+1$ ($a = 3$, $b = 3$, $d = 0$). Seviye i'de 3^i işlem vardır, ağaç $\log_3 n$ seviyede biter. Toplam $\sum_{i=0}^{\log_3 n} 3^i$. $T(n) = 2T(n/2) + 1$ gibi seçimler de kabul edilir.
- Sınav sorusu: tümevarımla $T(n) = 2T(n-1) + 2$ için $T(n) = O(n)$ mi? (ekrandan, 00:10:00)
  - Varsayım: $T(n-1) \le c(n-1)$. Yerine koyunca $T(n) \le 2c(n-1)+2 = cn-(-cn+2c-2)$.
  - $T(n) \le cn$ için $-cn+2c-2 \ge 0$ gerekir; bu yalnız $n < 2$ için sağlanır. Büyük n için tutmaz: $T(n) \notin O(n)$.
- Sınav sorusu: amortize maliyet (ekrandan, 00:13:10)
  - i 2'nin tam kuvvetiyse $c_i = i$, değilse $c_i = 1$. Belirli bir analiz yöntemi gerekmez; amortize maliyetin tanımı yeterlidir: toplam maliyet / işlem sayısı.
  - $T(n) \le \sum_{i=1}^{\lg n} 2^i + \sum_{i=1}^{n} 1 \le \frac{2^{\lg n+1}-1}{2-1} + n = 2n + n - 1 \approx 3n$. Kullanılan formül: $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$.
- Sınav sorusu: hiç çakışma olmama olasılığı (ekrandan, 00:16:50)
  - n anahtar, $m = 2n$ yuva, düzgün dağılım. Farklı çift sayısı $\binom{n}{2} = n(n-1)/2$; bir çiftin çakışma olasılığı $1/m$.
  - Hiç çakışma olmaması: $\left(1 - \frac{1}{m}\right)^{n(n-1)/2}$, burada $m = 2n$ yerine konur.
- İkili arama ağacı (binary search tree) (ekrandan, 00:26:25)
  - Desteklediği işlemler: arama, minimum, maksimum, öncül (predecessor), ardıl (successor), ekleme, silme. Hepsi ağacın yüksekliği h ile orantılıdır: $O(h)$.
  - Dengeli ağaçta $h = \Theta(\lg n)$. Elemanlar sıralı gelirse ağaç bağlı listeye döner ve $h = \Theta(n)$ olur.
  - Özellik: y, x'in sol alt ağacındaysa `y.key <= x.key`, sağ alt ağacındaysa `y.key >= x.key`.
  - Her düğüm `p` (parent), `left`, `right` işaretçilerini tutar. Çift yönlü bağ, silmede yapraktan köke doğru gitmek ve düğümün sol mu sağ mı çocuk olduğunu anlamak için gerekir.
- `TREE_SEARCH`, `TREE_MINIMUM`, `TREE_MAXIMUM` (ekrandan, 00:26:25 ve 00:30:00)
  - Arama özyinelemelidir: `x == NIL` ya da `k == x.key` ise `x` döner; `k < x.key` ise sola, değilse sağa gidilir.
  - Minimum: `x.left` NIL olana kadar sola gidilir. Maksimum: aynısı `right` ile.
- Ardıl ve öncül (ekrandan, 00:34:35)
  - Ardıl: x'ten büyük en küçük düğüm. Öncül: x'ten küçük en büyük düğüm. İkisi de silmede kullanılır.
  - `TREE_SUCCESSOR(x)` iki parçadır. (1) `x.right != NIL` ise `TREE_MINIMUM(x.right)` döner. (2) Değilse `y = x.p` ile yukarı çıkılır; `x == y.right` olduğu sürece `x = y`, `y = y.p`. Sonunda `y` döner: yukarı çıkarken ilk "sağa dönülen" ata.

  - Örnek ağaç (kök 15): $x = 13$ için $y = 7$; 13, 7'nin sağ çocuğu olduğu için $x = 7$, $y = 6$; 7, 6'nın sağ çocuğu olduğu için $x = 6$, $y = 15$; 6, 15'in sol çocuğu olduğu için döngü biter, ardıl 15'tir.
  - Karmaşıklık: iki durumda da en fazla ağaç yüksekliği kadar adım, $O(h)$.
- `TREE_INSERT(T, z)` (ekrandan, 00:41:10)
  - `x` kökten başlar, `y` bir önceki düğümü izler. `x` NIL olana kadar `z.key < x.key` ise sola, değilse sağa gidilir.
  - Döngüden sonra `z.p = y`. `y == NIL` ise ağaç boştur, `T.root = z`. Değilse `z.key < y.key` ise `y.left = z`, aksi hâlde `y.right = z`. Karmaşıklık $O(h)$.
- Silme ve `TRANSPLANT` (ekrandan, 00:47:10 ve 01:00:00)
  - Üç durum: z'nin çocuğu yok (bağ kaldırılır), tek çocuğu var (çocuk z'nin yerine geçer), iki çocuğu var (ardıl y bulunur ve z'nin yerine geçirilir).
  - `TRANSPLANT(T, u, v)`: u köküne sahip alt ağacı v köküne sahip alt ağaçla değiştirir. `u.p == NIL` ise `T.root = v`; `u` sol çocuksa `u.p.left = v`, değilse `u.p.right = v`; `v != NIL` ise `v.p = u.p`. `v`'nin çocuklarıyla ilgilenmez.
  - `TREE_DELETE` dört örnek durumla anlatıldı: S1 (sol çocuk NIL), S2 (sağ çocuk NIL), S3 (ardıl y, z'nin sağ çocuğu), S4 (ardıl y, sağ alt ağacın derininde). Satır 1-4 S1 ve S2'yi, satır 5-12 S3 ve S4'ü çözer.
  - Ardılın sol çocuğu her zaman boştur, çünkü ardıl sağ alt ağacın minimumudur.
  - Karmaşıklık: `TRANSPLANT` sabit zamandır; `TREE_MINIMUM` yüzünden toplam $O(h)$.
- Tekrar eden anahtarlar (ekrandan, 01:01:40)
  - Seçenekler: eşitler hep sağa (ya da hep sola) konur; her düğümde o anahtarın sayısı tutulur; düğümde bir bayrak tutulup eşitler sırayla bir sağa bir sola konur.
  - Sayaç tutmak ağacı sığ tutar ama düğümde ek bilgi ister; bayraklı yöntem ağacı daha dengeli tutar. Seçim uygulamaya bağlıdır.
- Radix ağaçları (radix tree) (ekrandan, 01:22:10 ve 01:28:50)
  - İkili dizgileri (bit strings) ya da karakter dizgilerini tutar; sözlük (dictionary) gerçeklemeleri için uygundur.
  - Bitte 0 sola, 1 sağa gidilir. Beyaz düğüm kümedeki bir dizgiyi tutar; gri düğüm boştur, yalnız yol için vardır.
  - Örnek küme: 1011, 10, 011, 100, 0. İkili arama ağacından farkı: bir küme için yapı tektir, ekleme sırasına bağlı değildir.
  - Karakter dizgilerinde düğümün en fazla alfabe boyutu kadar çocuğu olur (dallanma faktörü, branching factor).
  - Arama üç biçimde biter: beyaz düğümde (bulundu), gri düğümde (yol var ama dizgi sözlükte yok), NIL'de (yol yok).
  - Ekleme: düğüm varsa ve griyse beyaza çevrilir (örn. 101); yoksa yeni düğüm oluşturulur (örn. 11).
  - Silme: dizgi ara düğümdeyse düğüm griye çevrilir (örn. 0); yapraktaysa yaprak silinir ve geride kalan gri yapraklar özyinelemeli olarak kaldırılır (örn. 1011).
- Kırmızı-siyah ağaçlar (red-black tree) (ekrandan, 01:30:00)
  - Dengeyi garanti eden bir ikili arama ağacıdır; bütün işlemler $O(\lg n)$.
  - Beş kural: (1) her düğüm kırmızı ya da siyahtır; (2) kök siyahtır; (3) her yaprak (NIL) siyahtır; (4) kırmızı düğümün iki çocuğu da siyahtır; (5) her düğümden yapraklara giden bütün basit yollarda siyah düğüm sayısı aynıdır.
  - Siyah yükseklik $bh(x)$: x'ten bir yaprağa giden yoldaki siyah düğüm sayısı (x sayılmaz, NIL sayılır).
- Lemma: n iç düğümlü kırmızı-siyah ağacın yüksekliği en fazla $2\lg(n+1)$ (ekrandan, 01:40:00 ve 01:44:55)
  - x köklü alt ağaçta en az $2^{bh(x)} - 1$ iç düğüm vardır. Örnek: 10 (siyah), 7 (siyah), 3 (kırmızı), NIL. $bh(10) = 2$: $2^2 - 1 = 3$ düğüm. $bh(3) = 1$: $2^1 - 1 = 1$ düğüm.
  - Kural 4 yüzünden kökten yaprağa giden yolun en az yarısı siyahtır: $bh(\text{kök}) \geq h/2$.
  - $n \geq 2^{h/2} - 1 \Rightarrow \lg(n+1) \geq h/2 \Rightarrow h \leq 2\lg(n+1)$, yani $h = O(\lg n)$.
- Döndürme (rotation) (ekrandan, 01:44:55)
  - Sol döndürme x üzerinde: x'in sağ çocuğu y alt ağacın yeni kökü olur, x onun sol çocuğu olur, y'nin sol alt ağacı x'in sağ alt ağacına geçer. Sağ döndürme bunun tersidir.
  - Döndürme yalnız işaretçileri değiştirir; ağacın sıralı (inorder) dolaşımı değişmez.
- `RB_INSERT` (ekrandan, 01:50:00)

  - Normal ikili arama ağacı eklemesiyle aynıdır; sonra `z.left = z.right = NIL`, `z.color = RED` yapılır ve `RB_INSERT_FIXUP(T, z)` çağrılır.
  - Yeni düğüm hep kırmızı eklenir; siyah eklense kural 5 hemen bozulurdu.
  - Örnek (4 ekleniyor): Durum 1'de 4 ve ebeveyni 5 kırmızı, amca 8 kırmızı; 5 ve 8 siyaha, 7 kırmızıya boyanır. Durum 2'de 7 ve ebeveyni 2 kırmızı; 2 üzerinde sol döndürme yapılır. Durum 3'te 11 üzerinde sağ döndürme ve yeniden boyama yapılır, kök 7 olur.
  - Karmaşıklık: ekleme $O(\lg n)$, düzeltme $O(\lg n)$; toplam $O(\lg n)$.
- B-ağaçlarına giriş (B-tree) (ekrandan, 01:56:40 ve 02:00:00)
  - Disk gibi ikincil bellekler için tasarlanmış dengeli arama ağaçlarıdır; G/Ç (I/O) sayısını en aza indirmede kırmızı-siyah ağaçlardan iyidir, veritabanlarında kullanılır.
  - Düğümün çok sayıda çocuğu olabilir. `x.n` anahtarlı iç düğümün `x.n + 1` çocuğu vardır. Örnek: kökte M; altında D H (3 çocuk) ve Q T X (4 çocuk).
  - Düğüm genişliği disk sayfası (page) boyutuna göre seçilir. Kökte 1000 anahtar varsa 1001 çocuk olur; yüksekliği 2 olan ağaçta yaprak seviyesinde $1001^2 = 1\,002\,001$ düğüm ve $1\,002\,001\,000$ anahtar vardır.

Hocanın Özellikle Vurguladığı Kısımlar

- Ana teoremi tersten uygulama sorusunda somut sayılar
  - a, b, d harfleriyle çizilen ağaç kabul edilmedi; $a > b^d$ koşulunu sağlayan somut değerlerle bağıntı yazılmalıydı.
- Amortize maliyet tanımı
  - Amortize maliyet için her zaman toplam, muhasebe ya da potansiyel yöntemi gerekmez; maliyetler verildiyse toplamı işlem sayısına bölmek yeterlidir. Katsayılar önemsizdir; istenen mertebedir.
- İkili arama ağacında bütün işlemler $O(h)$
  - Ağaç dengeli değilse h, n'ye çıkar. Dengeli ağaç türleri (kırmızı-siyah, B-ağacı) bunu önlemek için vardır.
- Silmede ardılın sol çocuğunun neden boş olduğu
  - Ardıl sağ alt ağacın minimumudur; minimumun sol çocuğu olamaz. Hoca bunu sınıfa soru olarak sordu.
- Kırmızı-siyah ağaçta her değişiklikten sonra beş kural denetlenir
  - Kural 5 dengeyi sağlayan kuraldır. Yeni düğüm hep kırmızı eklenir.
- B-ağaçlarında disk erişimi
  - Çalışma zamanını disk okuma ve yazma sayısı belirler. Yüksekliği 2 olan ağaçta herhangi bir anahtara 2 erişimle ulaşılır.

Kısa Tekrar Notları

- $a > b^d \Rightarrow T(n) = \Theta(n^{\log_b a})$; örnek $T(n) = 3T(n/3) + 1 = \Theta(n)$.
- $T(n) = 2T(n-1) + 2$ için $T(n) \le cn$ tümevarımı yalnız $n < 2$ için tutar; $T(n) \notin O(n)$.
- $c_i = i$ (2'nin kuvvetleri), 1 (diğerleri): toplam $\approx 3n$.
- m yuvaya n anahtar: hiç çakışma olmaması $(1 - 1/m)^{n(n-1)/2}$.
- İkili arama ağacı: arama, min, max, ardıl, öncül, ekleme, silme hepsi $O(h)$.
- Ardıl: sağ alt ağaç varsa onun minimumu; yoksa yukarı çıkarken ilk sağa dönülen ata.
- Silme: 0 çocuk, 1 çocuk, 2 çocuk (ardıl yerine geçer). `TRANSPLANT` alt ağaç yer değiştirir.
- Tekrar eden anahtar: hep sağa, sayaç, ya da sağ-sol bayrağı.
- Radix ağacı: 0 sola, 1 sağa; beyaz = kümede, gri = boş; yapı ekleme sırasından bağımsız.
- Kırmızı-siyah ağaç: 5 kural; $h \le 2\lg(n+1)$.
- Döndürme: sol ve sağ, birbirinin tersi; inorder sırası korunur.
- RB ekleme: kırmızı ekle, fixup ile yeniden boya ve döndür; $O(\lg n)$.
- B-ağacı: `x.n` anahtar, `x.n + 1` çocuk; düğüm boyutu disk sayfasına göre.

Detaylı Açıklamalar

- Ders, sınav sorularının çözümüyle başladı. İlk soru ana teoremin tersten uygulanmasıydı. $T(n) = aT(n/b) + O(n^d)$ bağıntısında sonuç a, b ve d'nin ilişkisine göre değişir; $n^{\log_b a}$ sonucu $a > b^d$ durumunda elde edilir. Hoca $T(n) = 3T(n/3) + 1$ örneğini seçti: kök 1

işlem yapar ve 3 alt probleme ayrılır; ikinci seviyede 3, üçüncü seviyede 3^2 işlem vardır. Ağaç $\log_3 n$ seviyede biter ve toplam $\sum_{i=0}^{\log_3 n} 3^i$ olur. Hoca toplamın açılmasını beklemedi; doğru bağıntıyı kurmak ve ağacı somut sayılarla çizmek yeterliydi.

- İkinci soruda $T(n) = 2T(n-1)+2$ için $T(n) = O(n)$ olup olmadığı tümevarımla sınandı. $T(n-1) \leq c(n-1)$ varsayılır ve yerine konur: $T(n) \leq 2c(n-1)+2 = 2cn-2c+2 = cn+cn-2c+2$. Bunu $T(n) \leq cn - (\text{kalan})$ biçimine getirmek için kalan $-cn+2c-2$ olur; eşitsizliğin tutması için bu terimin negatif olmaması gerekir. $c > 0$ olduğundan bu yalnız $n < 2$ için mümkündür. Büyük n için tutmadığından $T(n)$, $O(n)$ değildir (ekrandan, 00:10:00).
- Üçüncü soru amortize maliyetin tanımını soruyordu. Bir dizi işlemin toplam maliyeti işlem sayısına bölünür. Burada maliyetler verilmişti: 2'nin kuvveti olan indislerde i, diğerlerinde 1. Toplam $\sum_{i=1}^{\lg n} 2^i + \sum_{i=1}^{n} 1 \leq 2^{\lg n+1} - 1 + n = 3n - 1$ civarındadır (ekrandan, 00:13:10).
  - Ders sonrası not: Tahtada sonuç $\approx 3n$ yazılıp yanına $\Theta(n)$ konmuştur; bu, n işlemin toplam maliyetidir. İşlem başına amortize maliyet $3n/n$, yani $O(1)$'dir (Ders 8'deki dinamik tablo sonucuyla aynı). Python ile sayıldığında $n = 1024$ için gerçek toplam $3060 \leq 3n = 3072$ çıkar.
- Dördüncü soru hash tablosunda hiç çakışma olmama olasılığıydı. n anahtardan $\binom{n}{2}$ farklı çift çıkar; bir çiftin aynı yuvaya düşme olasılığı $1/m$, düşmeme olasılığı $1 - 1/m$'dir. Bütün çiftlerin çakışmaması için bu olasılıklar çarpılır: $(1-1/m)^{n(n-1)/2}$. Soruda $m = 2n$ verildiği için bu değer yerine konur (ekrandan, 00:16:50).
- İkili arama ağaçlarında işlemlerin hepsi ağaç yüksekliğiyle orantılıdır. Hoca, düğümün içine yazılan sayının düğümün kendisi değil anahtarı (key) olduğunu hatırlattı. Arama kökten başlar ve her adımda anahtarı karşılaştırıp sola ya da sağa iner. Minimum için hep sola, maksimum için hep sağa gidilir.
- Ardıl, silme işleminin temelidir. Örneğin 16'yı silmek için ardılı 17 onun yerine taşınır (ekrandan, 00:30:00). Ardıl bulmada iki durum vardır. Sağ alt ağaç doluysa ardıl onun en küçük elemanıdır (örnekte 6'nın ardılı 7). Sağ alt ağaç boşsa ağaçta yukarı çıkılır: sola doğru her yukarı adımda kendinden küçük bir ataya gelinir; ilk kez sağa dönüldüğünde gelinen ata ardıldır (örnekte 13'ün ardılı 15). Öncül simetriktir: 6'nın öncülü 4'tür.
- Ekleme, aramaya benzer ama sona kadar iner. `y` her zaman `x`'in bir önceki düğümünü tutar; `x` NIL olduğunda `y` yeni düğümün ebeveynidir. `y` NIL ise ağaç boştur ve yeni düğüm kök olur.
- Silmede `TRANSPLANT` yardımcı fonksiyonu kullanılır. Bu fonksiyon u'nun yerine v'yi koyar: ebeveynin sol ya da sağ işaretçisini v'ye çevirir ve v NIL değilse v'nin ebeveynini günceller. `TREE_DELETE` kodu (ekrandan, 01:00:00):

```
TREE-DELETE(T, z)
1   if z.left == NIL
2       TRANSPLANT(T, z, z.right)
3   elseif z.right == NIL
4       TRANSPLANT(T, z, z.left)
5   else y = TREE-MINIMUM(z.right)
6       if y.p != z
7           TRANSPLANT(T, y, y.right)
8           y.right = z.right
9           y.right.p = y
10      TRANSPLANT(T, z, y)
11      y.left = z.left
12      y.left.p = y
```

- S3'te ardıl y doğrudan z'nin sağ çocuğudur; satır 6'daki koşul sağlanmaz ve doğrudan satır 10'a geçilir: y, z'nin yerine geçer, z'nin sol alt ağacı y'nin sol alt ağacı olur.
- S4'te y sağ alt ağacın derinindedir. Önce y'nin sağ çocuğu x, y'nin yerine geçirilir (satır 7), sonra y'nin sağına z'nin sağ alt ağacı bağlanır (satır 8-9), en son y, z'nin yerine geçer (satır 10-12).
- Radix ağacında yol, dizginin karakterlerinden kurulur. 0 ve 01 gibi önekler ağaçta bir düğümle temsil edilir; kümede yoksa düğüm gri kalır. Hoca bunu sözlük benzetmesiyle anlattı: sözlükte "araba" varken "ara" yoksa, "ara" için yol vardır ama düğüm gridir.

- Kırmızı-siyah ağacın yüksekliği için hoca önce bir alt ağaçta en az $2^{bh(x)} - 1$ iç düğüm olduğunu küçük bir örnekle gösterdi, sonra kural 4'ten dolayı herhangi bir kök-yaprak yolunun en az yarısının siyah olduğunu kullandı. Böylece $n \geq 2^{h/2} - 1$ ve $h \leq 2\lg(n+1)$ bulunur.
  - Ders sonrası not: Hoca bir yerde "her kırmızı için iki tane kırmızı ortaya çıkıyor" dedi (01:38:36). Doğrusu kural 4'tür: kırmızı düğümün iki çocuğu da siyahtır; bu yüzden bir yolda iki kırmızı art arda gelemez.
- Ekleme sonrası düzeltmede renk değiştirme ve döndürme birlikte kullanılır. Hoca fixup kodunu yazmadı; kodun ders kitabında bulunduğunu söyleyip işleyişi kitap şekli üzerinden anlattı (ekrandan, 01:50:00).
- B-ağaçlarında düğümler disk sayfası büyüklüğündedir. Bir düğümde 1000 anahtar varsa 1001 çocuk olur. Birinci seviyede 1001 düğümde $1\,001\,000$ anahtar, ikinci seviyede $1\,002\,001$ düğümde $1\,002\,001\,000$ anahtar bulunur (ekrandan, 02:00:00).
  - Ders sonrası not: Hoca yüksekliği 2 olan bu ağacın "bir milyondan fazla" anahtar tuttuğunu söyledi. Tahtadaki sayılara göre yalnız yaprak seviyesinde $1\,002\,001\,000$ anahtar vardır; toplam $1\,003\,003\,000$, yani bir milyardan fazladır (python ile doğrulandı).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.